

Classification Machine Learning on Histology Data

Task:

To evaluate the CRCHistoPhenotypes data, focusing on understanding the contributing attributes surrounding colon cancer, using multiple machine learning models for comparison to ultimately classify images and predict whether a cell is cancerous and its cell type, with an evaluation framework to compare the built models.

Goals:

- Create machine learning models to determine whether a cell is cancerous or not
- Create machine learning models to classify the cell type
- To evaluate the performance of the different models using an evaluation framework
- To select a final model that achieves the minimum scores of 0.9 for the cancer task and 0.6 for the cell task.

Prediction target:

This task focuses on two target attributes: `isCancerous` and `cellType`. `isCancerous` is a binary 0 or 1 attribute where 1 indicates cancerous, and 0 indicates not cancerous. The aim here is to correctly separate patients who have or don't have cancer based on images. The second target variable, `cellType`, aims to classify the type of cell as fibroblast, inflammatory, epithelial or others. Being able to classify these two target variables from the images is vital. The `isCancerous` attribute works to determine whether a cell may be linked to colon cancer, which is important for cancer detection. `CellType`, on the other hand, provides more details about the specific type of cell present in the tissue sample. Together, these two feature attributes allow us to perform both cancer detection and cell-type identification.

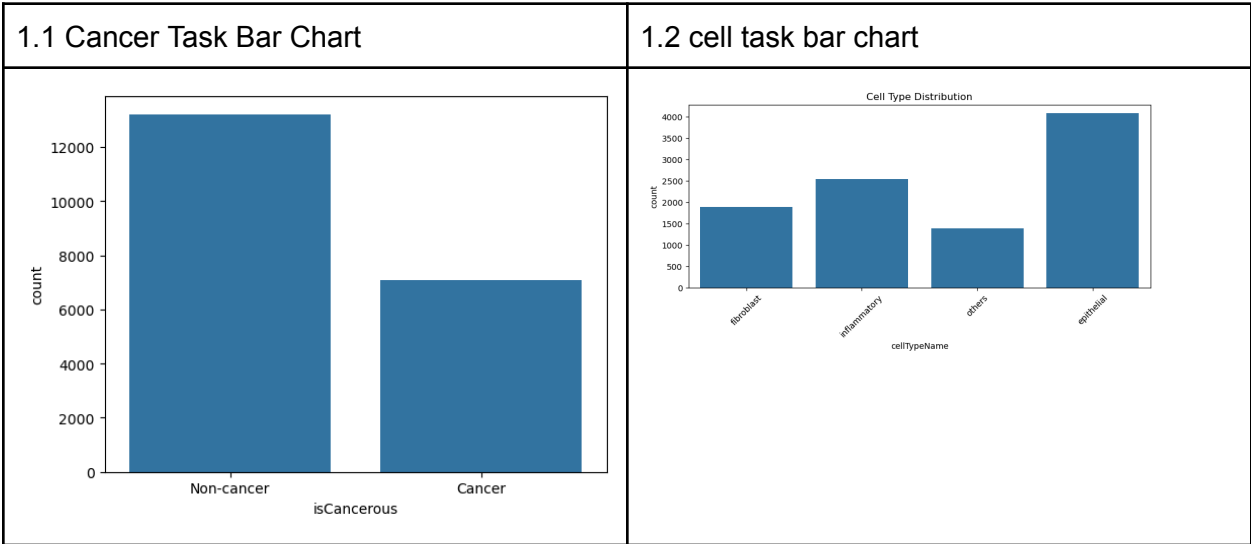
Dataset Description:

We have two different data sets: main and extra, which surround the field of histology, which is the study of cells. The datasets contain images that represent a microscopic view of tissue and cells. For our two tasks, the dataset used differed; for the `isCancerous` task, we used a combination of both main and extra. For the cell type task, we used only data from the main set. We labelled these datasets as `cancerous_df` and `cell_df`. Breaking down our main dataset, it contains samples from 60 different patients and includes annotated labels for both cancer classification and cell type classification. The extra dataset contains another set of samples from 39 patients, but they are only labelled for cancer status and do not include cell type information. Ultimately, the two datasets support our two different tasks, being a binary classification task for detecting cancerous cells and a multi-class classification problem for identifying specific cell types. Looking at the size of the dataset, main contains 6 attributes and 9896 records, extra contains 4 attributes and 10384 records. For the cancerous task or `cancer_df`, the number of records is 20280.

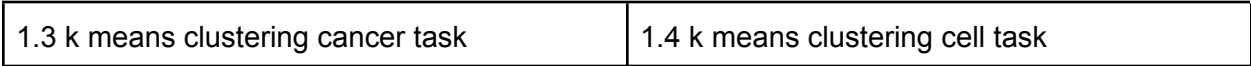
Exploratory Data Analysis (EDA):

The first part of EDA was looking at the feature variables. Figure 1.1 shows a bar plot with the frequency of cancerous and non-cancerous cases using the `cancerous_df` dataset. We have

two classes, making this a binary classification task; however, notably, we have a big class imbalance with non-cancer appearing far more often. This can pose a challenge where the model may be prone to predicting non-cancer due to seeing it more often; measures of class balancing will need to be considered. Figure 1.2 also shows a bar plot, but for cell tasks, We have 4 different classes: fibroblast, inflammatory, other and epithelial. Here, we also witness class imbalance, where epithelial is seen far more than the other 3 classes. Adding weights to classes may help our models later on.



To further visualise and understand the data, we created a simple k-means clustering model to depict the different classes. It is noted that we did not fine-tune this model as it is not one of our models we aim to evaluate and make predictions off. Figure 1.3 shows two charts. The one on the left shows the actual points, with yellow indicating one class and purple indicating the other. We do see some clear separation, with the majority of the yellow points being on the left and purple on the right. The plot on the right shows our k-means predictions, which crested 2 k values roughly centred in the two clusters created. Our model created two clear clusters for cancer and non-cancerous cells; however, when comparing the actual data, we can see that k-means could not separate the overlapped data points well. Figure 1.4 looks at cell tasks. We see that the actual data is far more mixed than the cancerous task; thus, for our future models, we can expect to get higher scores for cancerous tasks and lower scores for cell tasks due to the increased difficulty of separation. The k-means model did separate into 4 different clusters; however, the performance was not good.



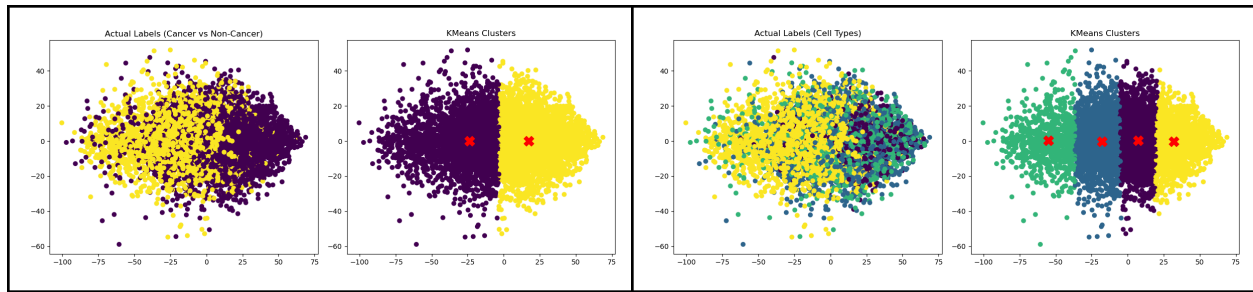
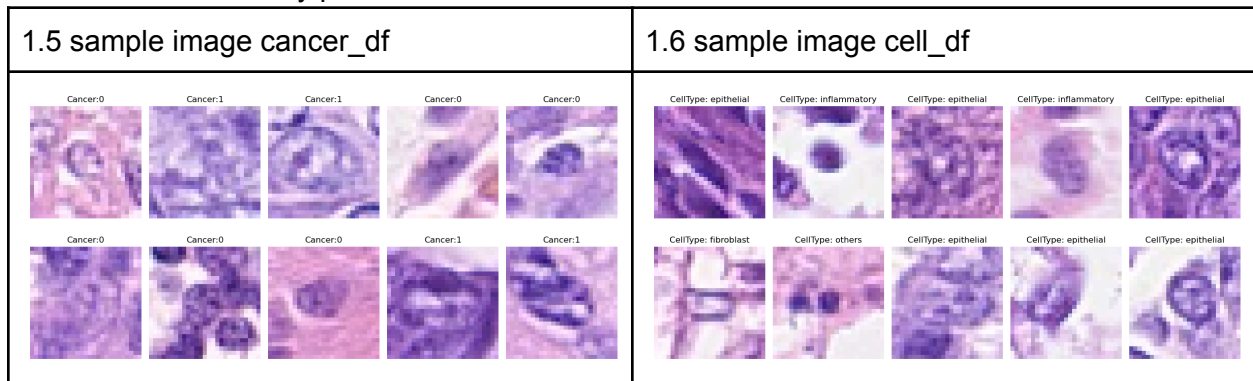


Figure 1.5 shows 10 images selected from the cancerous df, and Figure 1.6 shows another ten from the cell df set. These images are 27 by 27 pixels in size and can pose a future challenge, where many models prefer 32 by 32 or 64 by 64 pixels in size, preferring numbers that scale by the power of 2. Both sets of images show a similar colour scheme of pink, white and purple created by the stains the histologists used. This makes the cell easier to read, but also reduces the separation of colour between cells. The cell or tissue varies in different sizes and depths of colour and will be a key part in classification.



Data Pre-processing:

For basic data preprocessing, we first checked missing and duplicate values, but found none for both datasets. Secondly, we moved into the images themselves, as previously mentioned, the images, which were 27 by 27 pixels were transformed into 32 by 32 to make it easier, in particular, for our neural network model to read. A reproducibility code using random state 42 was also created; this is to ensure that all results are reproducible and, for fairness, all results, including splitting, were derived from random state 42. This supports ethics and responsibilities as data sets, but also allows us to fine-tune our models without constant random variables. In terms of colour shown by figures 1.6 and 1.5, the colours were fairly uniform and, as such, left them as is.

Splitting:

When splitting, we used the hold-out validation method, where we created a train set, a validation set and a test set derived from the validation set. This allows us to train our machine learning models on 60% of the data and 20% for validation and fine-tuning, whilst leaving another 20% of unseen data for our final test runs. This split focuses on our hyperparameter tuning to improve the validation test results, whilst leaving a final run to build our evaluation framework on. Testing on unseen data reflects ethical and responsible practices, working to

ensure fitness and no bias in the test results. For our cancer task, our split was Train: 16324, Val: 3908 and Test: 3956. For our cell type task, Train: 8057, Val: 2386 and Test: 1839. This split was derived from patients, where we considered the fact that different patients may have more or fewer pictures.

Model Development:

Three different models were created, each being used for both cancer and cell tasks. The first is our neural network model. A neural network model is a form of machine learning based on the structure of the human brain and the neurons within. It is made up of many layers that work to learn patterns from data by adjusting weights or parameters. This is done during the training phase, where the neural network model iterates over different batches of data, trying to learn the patterns and improve results on its own. During the second phase, the model uses what it has learned to make predictions.

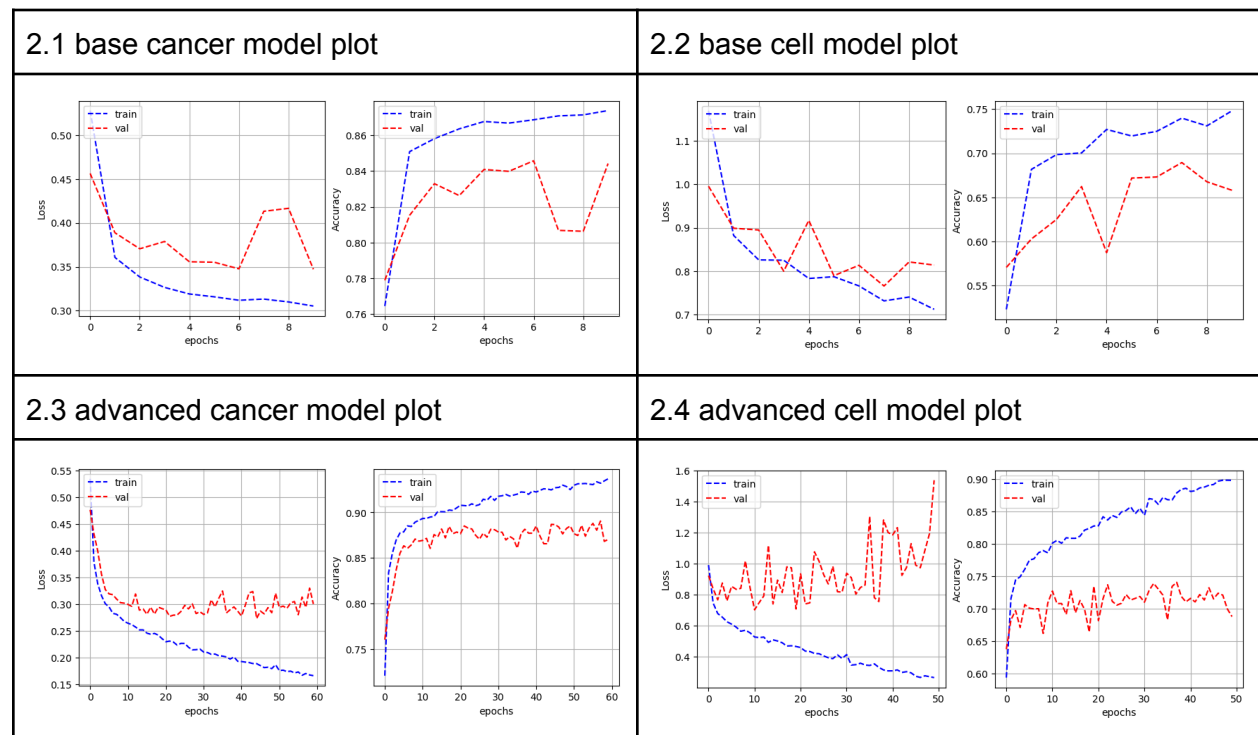
In this project, we use a type of neural network called a Convolutional Neural Network or CNN, which serves as our base model. CNNs were chosen as a base model as it can automatically learn spatial features such as edges, shapes, and textures without manual feature engineering. Also, compared to other neural network models it was simple and less computationally expensive. CNNs operate by sending an image through multiple layers that each try to extract more and more complex features. This layer is called a convolutional filter or layer. These findings are then combined to form these small findings that are small and can include shape, size, and borders. After which, the models combine these findings to form deeper, more complex relationships, which make strong predictions

Our second model, or the advanced model, we decided to switch to VGGNet, a deep CNN architecture. The model, similar to our base model, uses convolution filters but stacks multiple convolution layers instead of using large filters, which increases depth gradually.

With our advanced model, we also included an advanced technique called transfer learning. The principle for this is that we trained our model on CIFAR on a different dataset and then applied what it has learned from that dataset to our cancer and cell type tasks. Overall, transfer learning aims to improve our model's performance. Our code uses the VGG-style model to train on the CIFAR-10 dataset. Next, we then use that information learned on our history dataset. This provides extra resources that the model can learn from.

This project required several areas of hyperparameter tuning to improve results. Each change deeply affects the results of the neural networks. One of the first hyperparameters we tested with our base model was learning rate. We tested $lr=1e-4$ or 0.0001 and $lr=1e-3$ or 0.00. After testing the two different tasks had better results with different learning rates, the cancer task did better with $lr=1e-4$, and the cell task did better with $lr=1e-3$. Learning rate is a big hyperparameter and directly impacts how well a model learns. A lower value can limit or slow down the learning, but is generally more stable, whereas a higher value could yield better results only if it were stable. Due to a base model, we didn't change too much, leaving more of the hyperparameter tuning for our advanced model.

For our advanced model, we tested the following parameters: epochs, dropout rate and optimiser. Starting with epochs, we tested different numbers of training epochs, an epoch being the number of times the model has processed the entire dataset. The values tested included 20, 30, 50, 60 and 100 epochs and found that increasing the number of epochs generally improved performance up to a point. A trade-off was that it often caused overfitting, where the training rate went up far faster than the validation. We had to look at the loss rate, which showed how little or much the model was improving. After testing all models, we made a chart that shows loss over epoch and another for accuracy over epoch. Figures 2.1 and 2.2 look at our base model and have only 10 epochs, which is very few. The graphs show big swings in the validation line. For our advanced model, looking at figure 2.3, the cancer task, we see very limited overfitting, and though the model plateaus out at around 15 epochs, we still see some slight improvement up to our selected 60 epochs. For cell tasks, looking at Figure 2.4, the validation accuracy line has big changes, sitting between 65 and 75 for accuracy. We append the epochs for this task at 50 to prevent further overfitting.



Another important hyperparameter was the dropout rate, which was linked to our training model. We tested with different values in the VGG model: 0.2, 0.3, 0.4, and 0.5. Dropout rate works to help reduce overfitting by randomly turning off some neurons during training. Since this was applied to our training function that was used for both cancer and cell tasks, this required a balance of results between the two models. It was found that 0.4 has the best overall results, with 0.3 not far behind; values higher and lower than these two dropped our performance to fall within the mid 0.7s for macro f1.

We also tested different optimisers and regularisation settings, such as Adam and AdamW, as well as different weight decay values. Weight decay helps prevent overfitting by penalising large weights in the model. However, it was found that no weight decay yielded the best results. A surprise finding was with Adam and adamW, where adamW is a newer released variant; however the model performed better on the original torch.optim.Adam model.

Overall, hyperparameter testing was important in finding the best-performing configuration for our neural network model, helping to improve the accuracy and F1-score on the validation and test sets.

The second model was a random forest, a decision tree, which is a form of supervised machine learning. It functions by making different decisions, which lead to different paths; these paths ultimately lead to one of many classification possibilities. In this case, for the cancer task, it predicts cancerous or non-cancerous and for the cell task, one of the four different cell types. Decision trees are also structured like a tree, starting small which splits further and further. The different parts of a decision tree are as follows: the root node, which is the starting point that contains the whole data and will be split from this. We also have branches, which are the lines that connect the different nodes to show the paths. Next are the internal nodes, which are points where the data splits after the root node. Lastly is the leaf node, which is the end of the tree where the final prediction is made. The flow starts at the root node. If yes goes to the left or no goes to the right. This takes us to the branches that lead to the internal nodes, where another yes or no decision is made. This process is repeated until the internal node reaches a leaf node. Thus, in summary, it splits the data until a decision is made. The machine model created uses a variation of decision trees called random forest. Random forest works the same as a simple decision tree, but instead creates multiple trees. Each of the different trees created looks at different random parts of the data, in which the results of each will then be correlated, and a vote will take place to select the best one, which will be used. This is a form of an ensemble and works to get better results compared to a simple decision tree.

The third model was SVM. Support Vector Machines, or SVM, are another form of supervised learning model that works to split the data into different groups according to the different classes. It draws a hyperplane, and the different points indicate which class. Simply put it separates groups or clusters where different patterns may group different points. The core idea is to find the best hyperplane, the one that creates the largest gap between the two classes. A key term for SVM is margin, which refers to the distance between the different classes, such as our fire intensity and the line/ hyperplane. The closest points to the hyperplane from each class are referred to as support vectors and have an impact on where the line is drawn. In this report, we explored RBF, one of many algorithms that draw the hyperplane. RBF, standing for Radial Basis Function, looks at the distance between points and essentially connects the ones in proximity as part of a class, similar to clustering.

Model type	Strength	weakness
------------	----------	----------

Random forest	Easy to understand and visualise Computationally inexpensive Works with both numerical and categorical data Uses ensembles	Prone to overfitting Not the best at unseen data Can struggle with complex relationships
SVM	Can use different kernels: linear, RBF, and polynomial Good in datasets with clear separation	Not good for very big datasets Not good for noisy data Choosing the right kernel and parameters
Neural networks	Automatically learns patterns, trains itself Works well with non-linear data	Needs a large dataset Computationally expensive

Each of the models explored above was selected for a different reason as each have their own strengths and weaknesses. Decision Tree focusing on our random Forest model was chosen because it can handle complex, non-linear data and improve accuracy by combining multiple decision trees to reduce overfitting. SVM, on the other hand, was selected because it is effective at finding clear boundaries between classes, especially in more complex datasets. A neural network, a high-level model, was chosen due to its ability to automatically learn patterns in data and perform well on large, complex datasets. Overall, three different methods so that we can compare and determine what is best for our two classification tasks.

Performance Analysis (Quantitative Analysis and Visualisation):

Our performance analysis involved looking at three different things for each model and both tasks. We focused on accuracy, macro F1, and a classification report. The results used for this evaluation were obtained using an unseen test set. Accuracy is the measure of the overall correctness of the model, where it calculates the ratio of correct predictions to the total number of predictions made. Macro F1 is based on the standard F1 Score, being a balance of precision and recall, but for each class independently calculates and then takes the unweighted average of those scores to get us the macro F1 score.

Starting with our neural network models, our advanced model for the cancerous classification task achieved a high score, obtaining an overall accuracy of 0.8951 and a macro F1 of 0.8706; our results fell short of the 0.9 request. Despite this, this was the highest macro F1 score we were able to get after the many hyperparameter tunings and still shows that our model is capable of making accurate predictions for the cancerous dataset. Looking at the classification report, we see that the accuracy and F1 score when predicting class 0, which is the dominant class, were very high, easily reaching over 0.9, but the model struggled in predicting the second class. This imbalance in performance results in our macro F1 scores dropping, as we can

compare this to our weighted F1 scores, which sit at 0.9. Figure 3.1 shows the classification report.

3.1 Neural Network Cancer Results				
Cancer Test Set Accuracy: 0.8951 Macro F1: 0.8706				
Classification Report:				
	precision	recall	f1-score	support
0	0.95	0.90	0.93	2909
1	0.77	0.87	0.81	1047
accuracy			0.90	3956
macro avg	0.86	0.89	0.87	3956
weighted avg	0.90	0.90	0.90	3956

For our cell type classification task, we expected a difference in performance due to the increased number of classifications being 4 groups, and the mixed data shown with the clustering plots. Figure 3.2 shows our F1 scores, accuracy and classification report. Our macro F1 score was 0.6094, which met our threshold requirement of 0.6. The accuracy was 0.6857 which is quite high compared to the F1 score. Similar to the neural network model, the performance differed in predicting each class, where the model performed far better for the majority class; class 0 had an F1 score of 0.83, making it a strong model in predicting this class; however, the scores for the remaining classes were lower. Referencing Figure 2.4, which showed the accuracy over the epochs, we see a fair bit of overfitting.

3.2 Neural Network Cell Results				
Cell Test Set Accuracy: 0.6857 Macro F1: 0.6094				
Classification Report:				
	precision	recall	f1-score	support
0	0.93	0.76	0.83	925
1	0.52	0.62	0.56	261
2	0.61	0.70	0.65	432
3	0.35	0.44	0.39	221
accuracy			0.69	1839
macro avg	0.60	0.63	0.61	1839
weighted avg	0.73	0.69	0.70	1839

Moving on to our second model type, the random forest decision tree, for the cancer classification task, our model fell short of the neural network model with a macro F1 score of 0.8315935204927742 and an accuracy of 0.872093023255814. These results still show that we were able to build a strong model with results showing it can accurately predict over 80% of the time. We see that the model was better at predicting class 0 with an F1 score of 0.91, but not too good at class 1 at 0.75.

3.3 Random Forest Cancer Results

```
cancer task
Accuracy: 0.872093023255814
Macro F1: 0.8315935204927742
      precision    recall  f1-score   support

     0       0.90      0.93      0.91      2909
     1       0.78      0.72      0.75      1047

   accuracy          0.87      3956
  macro avg       0.84      0.82      0.83      3956
 weighted avg       0.87      0.87      0.87      3956

[[2695  214]
 [ 292  755]]
```

For our cell type classification, random forest accuracy was 0.6460032626427407, and the macro F1 was 0.4430053126164465. The lacklustre F1 result showed that the model was not good at identifying the different cell types. This was specifically reinforced by looking at the classification report in Figure 3.4, where class label 3 was not identified at all. This shows that the random forest was unable to separate the minority classes well or at all. Random forest, however, did well in separating the majority class of class 0. The separation in performance between classes made the macro F1 score drop; however, the weighted F1 score did well, sitting at 0.6.

3.4 Random Forest Cell Results

```
cell task
Accuracy: 0.6460032626427407
Macro F1: 0.4430053126164465
      precision    recall  f1-score   support

     0       0.77      0.82      0.80      925
     1       0.71      0.25      0.37      261
     2       0.48      0.83      0.61      432
     3       0.00      0.00      0.00      221

   accuracy          0.65      1839
  macro avg       0.49      0.48      0.44      1839
 weighted avg       0.60      0.65      0.60      1839

[[763   3 159   0]
 [110   65  85   1]
 [ 58   14 360   0]
 [ 62    9 150   0]]
```

Our lost model type was SVM for the cancer task, our macro F1 score was 0.8422099821335516, and the accuracy achieved was 0.8677957532861477. The result again was below the 0.9 requirement. Class imbalance was the biggest problem again with this model, where it had an F1 score of 0.91 for class 0, but its class 1 F1 was 0.78.

3.5 SVM cancer results

```

Cancer Task
Accuracy: 0.8677957532861477
Macro F1: 0.8422099821335516
      precision    recall  f1-score   support

     0       0.95       0.86       0.91       2909
     1       0.70       0.88       0.78       1047

 accuracy
macro avg       0.83       0.87       0.84       3956
weighted avg       0.88       0.87       0.87       3956

[[2513  396]
 [ 127  920]]

```

For the cell task, SVM did well with a macro F1 score of 0.5820315840921085 and an accuracy of 0.6965742251223491. Looking at the accuracy, SVM had the highest accuracy of all the models for cell tasks, but the F1 score fell a little behind. The model was able to separate all 4 classes with the strongest capability for class 0, followed by class 2. If looking at weighted F1 results, the SVM model did well with a F1 Score of 0.7, making it a moderately strong model. This shows that the model did not predict class 3 well compared to the other classes, making its macro score go down.

3.6 SVM cell results

```

Cell Task
Accuracy: 0.6965742251223491
Macro F1: 0.5820315840921085
      precision    recall  f1-score   support

     0       0.88       0.82       0.85       925
     1       0.58       0.46       0.51       261
     2       0.55       0.80       0.66       432
     3       0.39       0.26       0.31       221

 accuracy
macro avg       0.60       0.58       0.58       1839
weighted avg       0.70       0.70       0.69       1839

[[758  28 114  25]
 [ 60 119  54  28]
 [ 20  29 346  37]
 [ 23  30 110  58]]

```

Advanced Techniques:

The CNN neural network model was selected for further tuning because the model structure was designed with the intention of image classification tasks. Unlike Random Forest and SVM, which treat image pixels as numerical features, CNNs can automatically learn visual patterns. Further, it was believed that the neural network had the biggest room for growth. The base model also produced fairly strong results, which gave this model the chance of getting the required macro F1 scores.

The advanced technique employed was transfer learning, which was selected as it can provide more training room beyond the histology dataset. Training a CNN model using our singular

dataset, as shown by the figures in our base model, predicted high levels of overfitting (Figures 2.1 and 2.2). We can reduce overfitting where the model can train on similar data.

To deploy this transfer learning we built a VGG-style CNN that we used on the CIFAR-10, another image dataset. With this, the model can learn some of the shapes, colours, and other patterns from images. These learned weights would then be transferred to our two histology classification tasks, finally replacing the final output layer and retraining the model on the cancer and cell-type datasets.

When comparing the results of transfer learning, our base model for the cell task produced a macro F1 score of 0.8376831022643269 and an accuracy of 0.8441658137154555. For cell type classification, it had a macro F1 of 0.5901956282723944 and an accuracy of 0.6580050293378038. Our advanced model for cancerous tasks had an F1 score of 0.8706 and an accuracy of 0.8951. For the cell, f1 was 0.6094, and the accuracy was 0.6857. This made a macro F1 improvement of 0.0329 for the cancer task, and for the cell task, we improved results by 0.019204.

Model Comparison:

By creating three different models, we are able to compare the different approaches and methods used for classification. The neural network model using CNN, and our advanced model with the VGG model combined with transfer learning, was built with the purpose of processing image data. Unlike the other two models, it was able to automatically learn the different image features directly from the histology images. The addition of transfer learning with CIFAR-10 further helped its ability to identify patterns by pulling information learned from a different dataset. This made the neural network model well-suited to both the cancer and cell type classification tasks. However, this came at a cost of computational power and time to run.

The random forest model performs classification by creating many decision trees and combining their predictions through voting. Random forests are generally used on structured tabular data, where they can better capture non-linear relationships and interactions between variables. However, using the histology data, which includes images, the random forest model may not have the means to read and analyse the pixels well. While the random forest was able to identify the dominant classes, it struggled to separate the more complex minority classes, as shown in the cell type classification task, where class 3 was not identified at all.

The SVM model more or less provided a balance between the two approaches, where it was able to find deeper patterns in the data whilst being less costly to run. Using the RBF kernel allowed the model to create non-linear decision boundaries. Compared to random forest, SVM was better able to distinguish between classes because it focuses on finding optimal boundaries between groups through the use of support vectors. However, the results are lower than those of a neural network, where SVM does not learn entire image features and instead relies on the extracted pixel information provided to it.

When comparing the results of the three models, the evaluation framework put the neural network with the highest F1 and accuracy results, followed by SVM coming second, and random forest with the weakest performance. These results were the same for both classification tasks.

Table of results:

model	Cell type task Macro F1	Cancerous task Macro F1	Cell type task Accuracy	Cancerous task Accuracy
Neural network	0.6094	0.8706	0.6857	0.8951
Random forest	0.4430	0.8316	0.6460	0.8721
SVM	0.5820	0.8422	0.6966	0.8678

Final Model Selection:

Cancer classification task: Support vector machines (RBF kernel)

Cell type classification task: Neural network (VGG, transfer learning)

Justification:

Although the neural network achieved slightly higher performance, the SVM achieved comparable results while requiring substantially less training time and computational resources. Given the relatively small performance difference and lower computational cost, SVM was selected as the preferred model for the cancer classification task. The SVM model only has base hyperparameters with no tuning for model performance, showing that the model has room for performance where we could add the class weights, transformation and training that was applied to the neural network needed to get a 0.03 difference in score. A macro F1 score of 0.8315935204927742 and an accuracy of 0.872093023255814 are still strong results. The reason for choosing this model is the potential support vector machines have in being able to be optimised and improving performance.

For the cell type classification task, we selected a neural network, this is as it gave the highest results with the only one meeting the macro F1 requirement of 0.6. The high F1 score, combined with its ability to better separate the model classes overall, made it a better choice compared to models like random forest, which couldn't predict class 3 at all. This choice was made purely based on the performance in our evaluation framework.

Overall, the application of transfer learning was able to boost performance. We see this due to the difference in results from our base and advanced models for both the neural network tasks. The spikes and the overfitting were greatly reduced after the technique shown, especially in the cancer task when comparing Figures 2.1 and 2.3. Aside from visuals, the performance of the F1 Score and accuracy went up after the technique was applied.

Ethics and Responsibilities:

One of the key challenges to ensuring we remain ethical and follow responsible practices lies in preventing data leakage. This is to ensure that the three data splits, train, validation and test, remain separate where the model does not mix them. When data leakage happens, this can cause biased and unfair results where the model has already been trained or seen the validation or test data, boosting the performance. To ensure fairness, here we utilised holdout validation splitting and used only the test set for the final results, keeping 20% of the data unseen.

A second point of ethical concern is the need to report findings as they are. Despite not having the best results, especially for our cancer task, which did not meet the required threshold, as analysts, our job is to report honestly about our results.

Evaluation:

Overall, our model did well in predicting the majority class. Having more class balancing or stronger weights would be an improvement to add in the future. We can include that we were able to create, compare, and evaluate three different models for our two classification tasks despite not meeting the requirements for cancer classification. Our cell type classification model met expectations. Our task and goals were met.

References:

vgg-pytorch. (2020, March 16). PyPI. <https://pypi.org/project/vgg-pytorch/>

GeeksforGeeks. (2018, October 3). Principal Component Analysis with Python. GeeksforGeeks. <https://www.geeksforgeeks.org/data-analysis/principal-component-analysis-with-python/>

scikit-learn. (2009). sklearn.decomposition.PCA — scikit-learn 0.20.3 documentation. Scikit-Learn.org. <https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>

PyTorch. (2023). PyTorch. Pytorch.org. <https://pytorch.org/>

VGG Net | Build VGG Net from Scratch with Python. (2021, June 11). Analytics Vidhya. <https://www.analyticsvidhya.com/blog/2021/06/build-vgg-net-from-scratch-with-python/>

GeeksforGeeks. (2017, December 23). Image Classification using CNN. GeeksforGeeks. <https://www.geeksforgeeks.org/machine-learning/image-classifier-using-cnn/>

Python Machine Learning. (n.d.). Wwww.w3schools.com. https://www.w3schools.com/python/python_ml_getting_started.asp

GeeksforGeeks. (2019, November 23). What is Transfer Learning? GeeksforGeeks. <https://www.geeksforgeeks.org/machine-learning/ml-introduction-to-transfer-learning/>

Transfer learning with a pretrained ConvNet | TensorFlow Core. (n.d.). TensorFlow. https://www.tensorflow.org/tutorials/images/transfer_learning

IBM. (2024, February 12). Transfer Learning. Ibm.com.

<https://www.ibm.com/think/topics/transfer-learning>

Javaheri, D. B. (2023, May 4). A hands-on tutorial for transfer learning in Python. Domino.ai;

Domino. <https://domino.ai/blog/transfer-learning-in-python>

AdamW vs Adam: Key Differences and Best Use Cases for Developers. (2025, September 19). Prodia.com.

<https://blog.prodia.com/post/adam-w-vs-adam-key-differences-and-best-use-cases-for-developers>